Pilhas e Filas

Estrutura de Dados

Universidade Federal de Juiz de Fora
Departamento de Ciência da Computação

Conteúdo

- Introdução
- Pilha
  - Implementação com vetor
  - Implementação encadeada
- Fila
  - Implementação com vetor
  - Implementação encadeada
- Exercícios

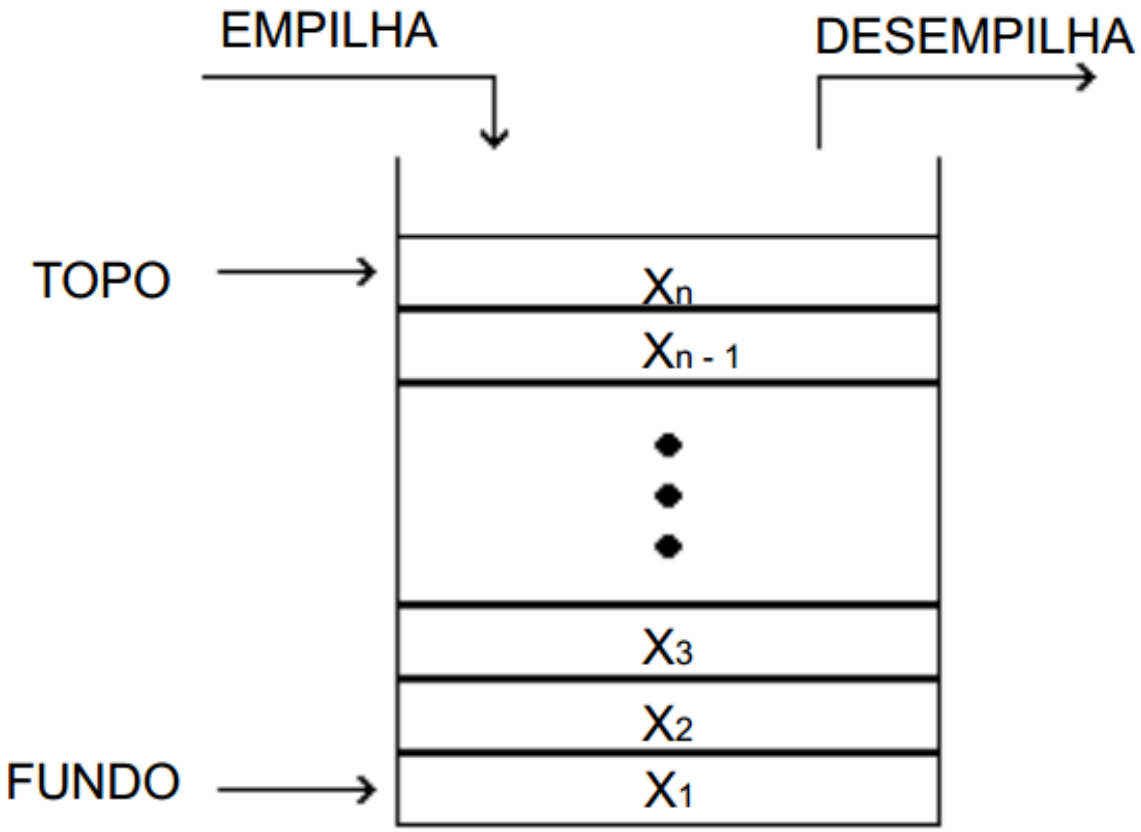
Introdução
Pilhas e Filas

- Pilhas e Filas são estruturas de dados utilizadas em várias aplicações.
- Pilhas e Filas são **listas** que possuem uma **disciplina de acesso** bem definida.
- Exemplos:
  - Pilha de pratos
  - Fila de banco

Pilhas

- A pilha é uma lista onde todas as inclusões e exclusões de nós são feitas em uma **única extremidade**.
- Essa extremidade é conhecida como **topo**.
- Usa-se o critério de retirar em primeiro lugar o último nó que foi incluído na lista.
- Este critério é conhecido como **FILO** (**F**irst **I**n **L**ast **O**ut).
- A ideia desta estrutura é a de uma pilha de pratos.

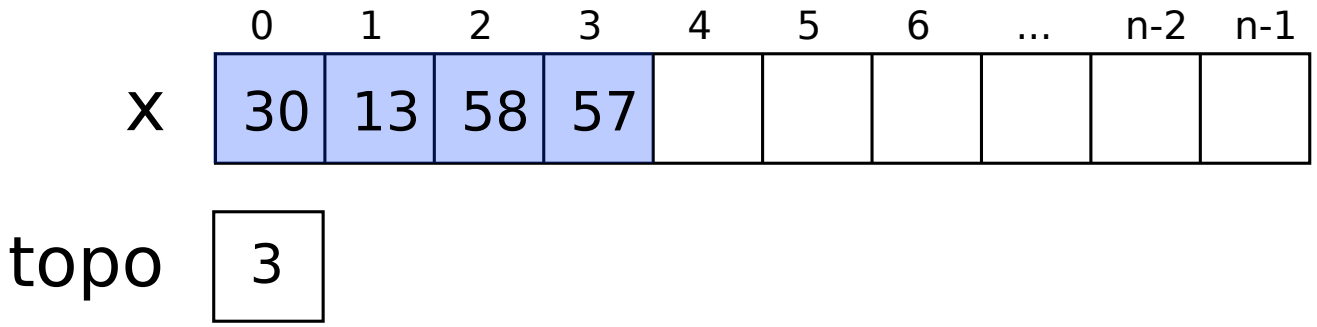
Pilhas



Pilhas

Representação com vetor

- Representação com vetor (ou contiguidade dos nós).
- Esquematicamente, uma pilha contígua com capacidade máxima para n nós pode ser representada da seguinte forma:



Pilhas

Representação com vetor

- Desta forma, o TAD `PilhaCont` para representar uma pilha de números reais, pode ser implementado como:

```
class PilhaCont
{
 private:
  int max;  // capacidade maxima
  int topo; // posicao do topo da pilha
  float *x; // vetor que armazena os dados da pilha

 public:
   PilhaCont(int tam);
   ~PilhaCont();
   float consulta();          // consulta o topo
   void empilha(float val); // insere No no Topo
   void desempilha();         // elimina No do Topo
   bool vazia();
};
```

Pilhas

Representação com vetor

```
PilhaCont::PilhaCont(int tam)
{
  max  = tam;
  topo = -1;
  x = new float[max];
}

PilhaCont::~PilhaCont()
{
  delete [] X;
}
```

- O construtor cria inicialmente uma pilha vazia.
- A pilha vazia é marcada com topo com o valor -1.
- Condição de pilha vazia: `topo == -1`.

Pilhas

Representação com vetor

```
float PilhaCont::consulta()
{
  if(topo != -1)
    return X[topo];
  else
  {
    cout << "Pilha vazia!" << endl;
    exit(1);
  }
}

bool PilhaCont::vazia()
{
  return (topo == -1);
}
```

Pilhas

Representação com vetor

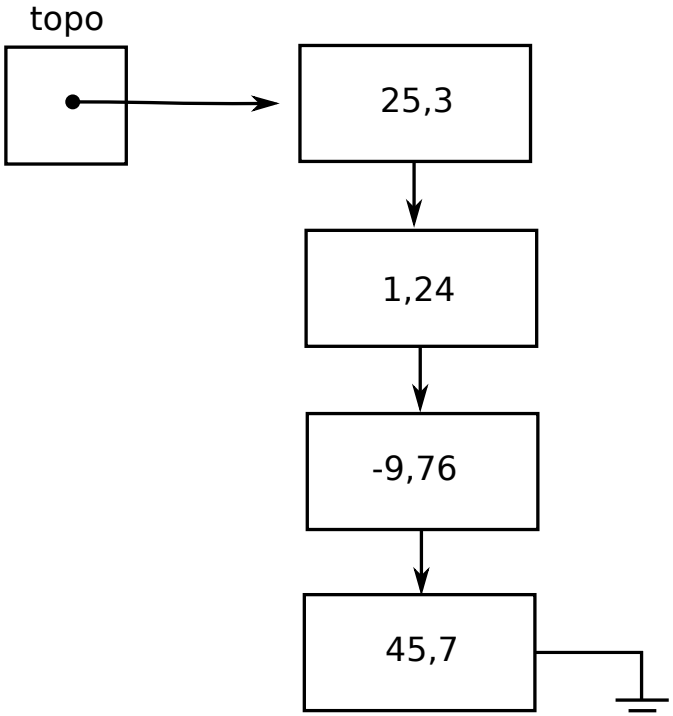
```
void PilhaCont::empilha(float val)
{
  if(topo == (max - 1)){
    cout << "Vetor Cheio!" << endl;
    exit(2);
  }
  topo = topo + 1;
  x[topo] = val;
}

void PilhaCont::desempilha()
{
  if(topo == -1){
    cout << "Pilha Vazia!" << endl;
    exit(3);
  }
  topo = topo -1;
}
```

Pilhas

Representação encadeada

- Uma pilha de valores reais com n nós pode ser **representada** através de **uma lista simplesmente encadeada**.



- Portanto, será necessário definir dois TADs: `No` e `PilhaEncad`.

Pilhas

Representação encadeada

- O TAD `No` possui a mesma implementação usada anteriormente para uma lista simplesmente encadeada.
- A única diferença é que agora o dado de cada nó é um valor real, representado por um `float`.

```
class No
{
 public:
   // ...

 private:
   float info; // informacao do No
   No *prox;
};
```

Pilhas

Representação encadeada

```
class PPilhaEncadilha
{
 public:
   PilhaEncad();
   ~PilhaEncad();

   float consulta();
   void empilha(float val);
   void desempilha();
   bool vazia();

 private:
   No * topo;
   int n;
};
```

Pilhas

Representação encadeada

```
PilhaEncad::PilhaEncad()
{
  topo = NULL;
}

float PilhaEncad::consulta()
{
  if(topo != NULL)
    return topo->getInfo();
  else
  {
    cout << "Pilha vazia!" << endl;
    exit(1);
  }
}
```

Pilhas

```
void PilhaEncad::empilha(float val)
{
  No *p = new No();
  p->setInfo(val);
  p->setProx(topo);
  topo = p;
}

void PilhaEncad::desempilha()
{
  No *p;
  if(topo != NULL)
  {
    p = topo;
    topo = p->getProx();
    delete p;
  }
  else
    rotina_erro(1);
}
```

Pilhas

Representação encadeada

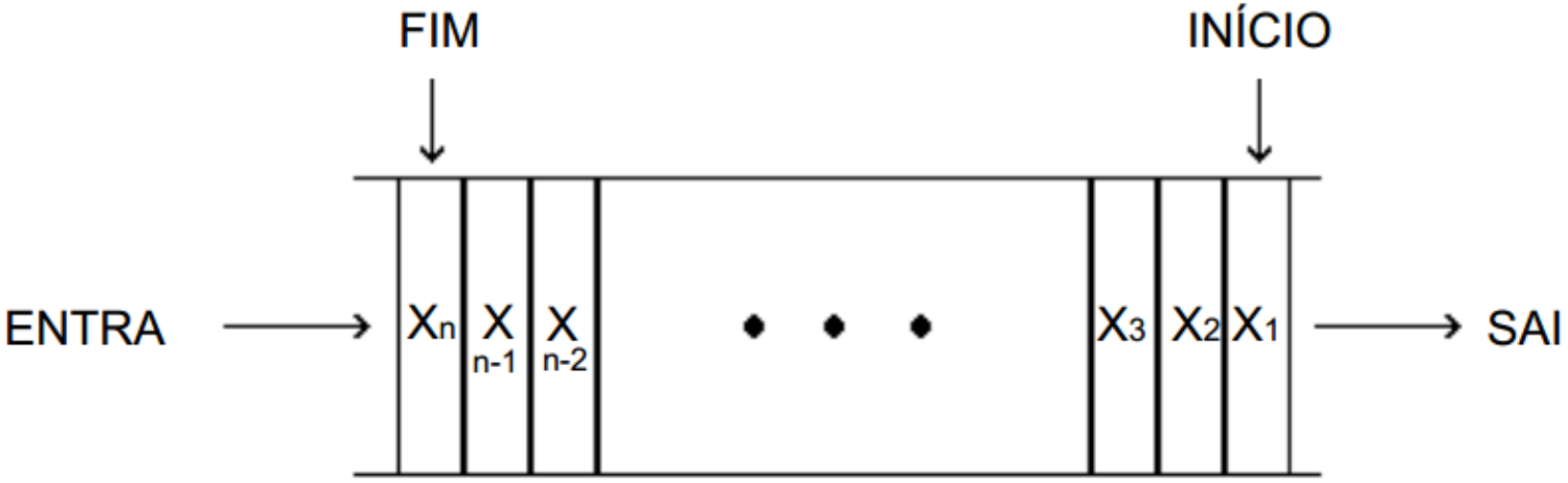
```
bool PilhaEncad::vazia()
{
  return (topo == NULL)
}

PilhaEncad::~PilhaEncad()
{
  No *p = topo;
  while(topo != NULL)
  {
    topo = p->getProx();
    delete p;
    p = topo;
  }
}
```

Filas

- Pilhas e Filas são listas que possuem uma disciplina de acesso bem definida.
- **Fila**:
  - A fila é uma lista onde todas as **inclusões são feitas em uma extremidade** e todas as **exclusões na outra extremidade**.
  - Usa-se o critério de retirar em primeiro lugar o primeiro nó incluído na lista.
  - Este critério é chamado de **FIFO** (**F**irst **I**n **F**irst **O**ut).
  - Esta estrutura é equivalente a uma fila de banco.

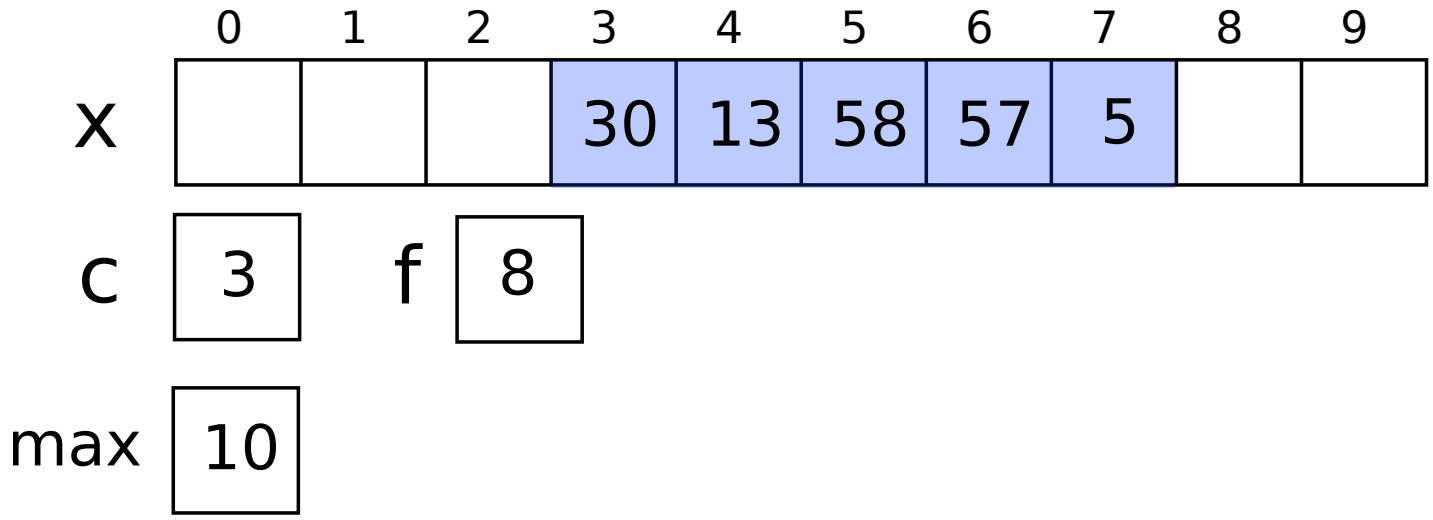
Filas



- Representações:
  - Contígua
  - Encadeada

Filas

- Representação por contiguidade dos nós.
- Uma fila contígua onde já ocorrem $(n+1)$ entrada e k saídas de nós, teria a seguinte representação esquemática:



- Sendo c o índice do vetor que contém o nó do começo e f o índice do nó que está no fim.

TAD Fila Contígua

```
class FilaCont
{
 private:
  int m;     // maximo de elementos do vetor
  int c;     // indices do No que esta no inicio
  int f;     // indices do No que esta no fim
  float *X; // vetor que armazena a fila

 public:
  FilaCont(int tam);
  ~FilaCont();
  float consultaInicio(); // consulta No do inicio
  void entra(float val);  // insere No no fim
  void sai();             // elimina No do inicio
  bool vazia();           // verifica se esta vazia
};
```

TAD Fila Contígua

- Construtor e destrutor

```
FilaCont::FilaCont(int tam)
{
    m = tam;
    c = 0;
    f = 0;
    x = new float[m];
}

FilaCont::~FilaCont()
{
    delete [] x;
}
```

TAD Fila Contígua

```
bool FilaCont::vazia()
{
    return (c==f);
}

float FilaCont::consultaInicio()
{
    if (!vazia())
        return x[c];
    else {
        cout << "Fila vazia" << endl;
        exit(1);
    }
}
```

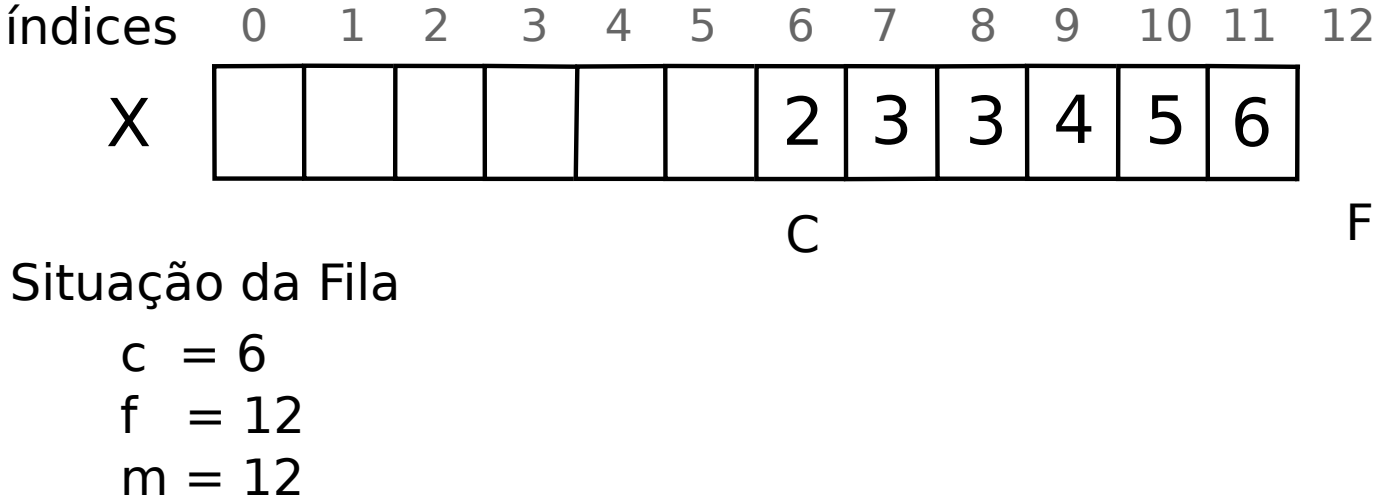
TAD Fila Contígua

```
void FilaCont::entra(float val)
{
    if(f == m) // fila cheia
    {
        cout << "Fila cheia" << endl;
        exit(1);
    }
    x[f] = val;
    f = f + 1;
}

void FilaCont::sai()
{
    if(!vazia())
    {
        c = c + 1;
    }
}
```

Problemas do TAD Fila Contígua

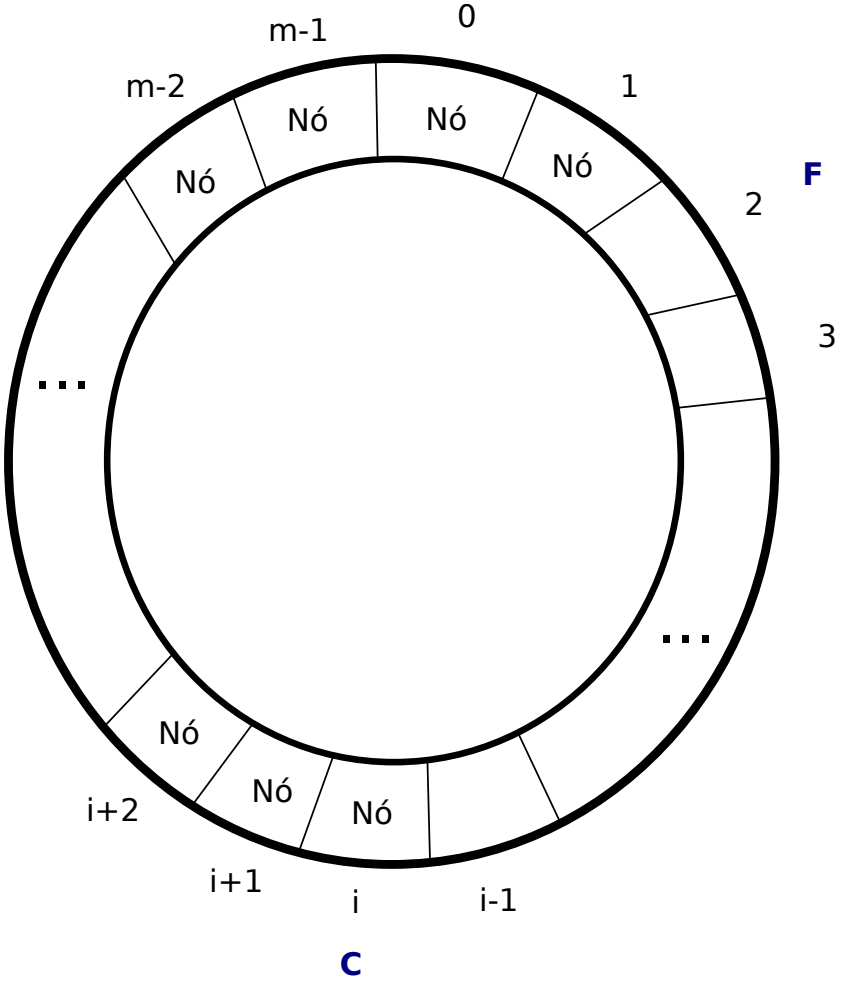
- O TAD `FilaCont` apresenta algumas restrições.



- Quando $f = m$ não é se pode mais inserir na fila.
- Entretanto, se $c > 0$ então existe espaço no início da fila.
- Esse espaço deveria ser usado para inserir mais nós na fila.
- A solução mais comum é utilizar uma representação circular do vetor que representa a fila.

Problemas do TAD Fila Contígua

- Ideia: todas as operações de incremento em c e f, quando sai e entra nó na fila, devem ser realizadas módulo m.
- TAD Fila Circular:

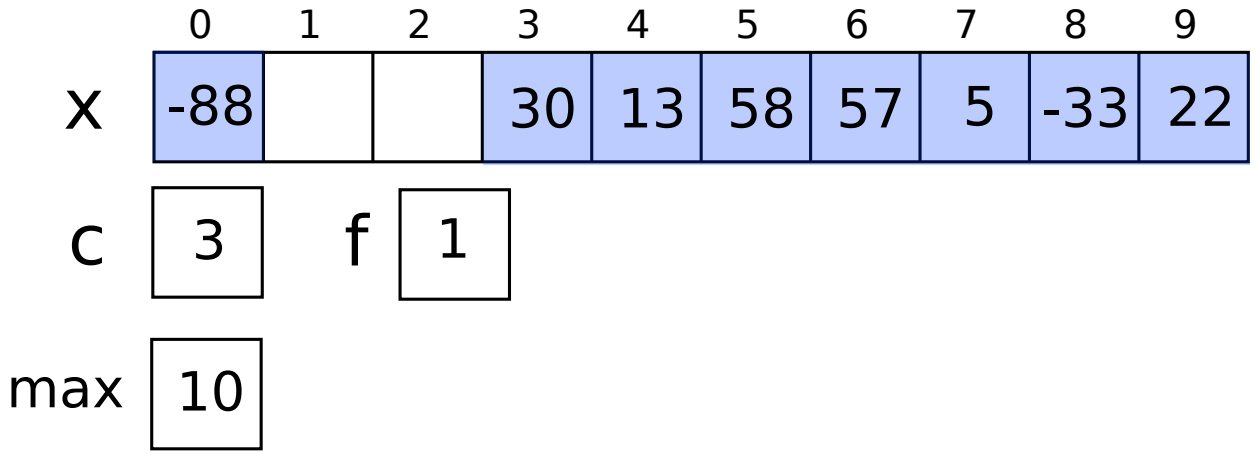


Situação da Fila

c = i
f = 2

Problemas do TAD Fila Contígua

- Exemplo:



- E se mais dois valores fossem inseridos na fila?
- Nesse caso, `f=c` e assim a condição de fila vazia se confundiria com a de fila cheia.
- Para evitar esse problema usa-se um contador `n` para saber quantos valores estão de fato na fila.

TAD Fila Circular

```
class FilaCirc
{
 private:
  int m;     // maximo de elementos do vetor
  int c;     // indices do No que esta no inicio
  int f;     // indices do No que esta no fim
  int n;     // atual numero de elementos do vetor
  float *X; // vetor que armazena a fila

  int inc(int ind); // incrementa indice

 public:
  FilaCirc(int tam);
  ~FilaCirc();
  float consultaInicio(); // consulta No do inicio
  void entra(float val);  // insere No no fim
  void sai();             // elimina No do inicio
  bool vazia();           // verifica se esta vazia
};
```

TAD Fila Circular

Incremento circular

- A operação de *incremento circular* para o TAD Fila Circular pode ser implementada da seguinte forma:

```
int FilaCirc::inc(int ind)
{
  if (ind == m-1)
    return 0;
  else
    return ind+1;
}

// ou

int FilaCirc::inc(int ind)
{
  return (i+1) % m;
}
```

TAD Fila Circular

Incremento circular

- Construtor: precisa ser modificado para incluir n, o número de nós presentes na fila.

```
FilaCirc::FilaCirc(int tam)
{
    m = tam;            // capacidade maxima
    n = 0;              // numero de nos na fila
    c = f = 0;          // comeco, fim
    x = new float[m]; // vetor com os dados
}

FilaCirc::~FilaCirc()
{
    delete [] x;
}
```

TAD Fila Circular

- Operação `consultaInicio()`: sem alterações com relação ao TAD anterior `FilaCont`.
- A condição de fila vazia agora é diferente.
- É preciso verificar o valor de n, que representa o número de nós inseridos na fila:
  - se $n = 0$: a fila está **vazia**;
  - se $n = m$: a fila está **cheia**, não é possível inserir mais nós.

```
bool FilaCirc::vazia()
{
  return (n==0);
}
```

- Para o TAD `FilaCirc` só é preciso mudar as operações de entrada e saída da fila usando o **incremento circular** .

TAD Fila Circular

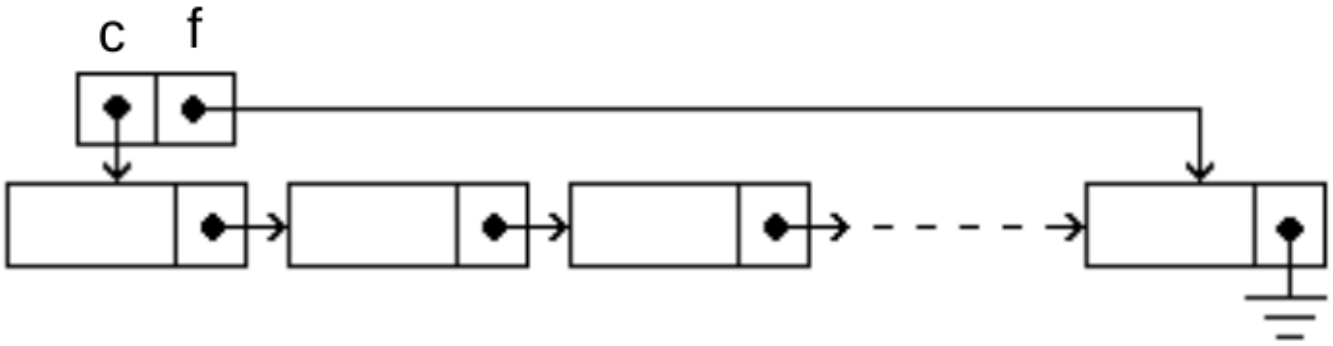
```
void FilaCirc::entra(float val)
{
    if(n == c) {
        cout << "Fila cheia" << endl;
        exit(1);
    }
    x[f] = val;
    f = inc(f);
    n = n + 1;
}

void FilaCirc::sai()
{
    if(!vazia()) {
        c = inc(c);
        n = n - 1;
    }
}
```

Fila

Encadeamento dos nós

- Representação por encadeamento dos nós.
- Esquematicamente, seja uma fila encadeada com n nós:

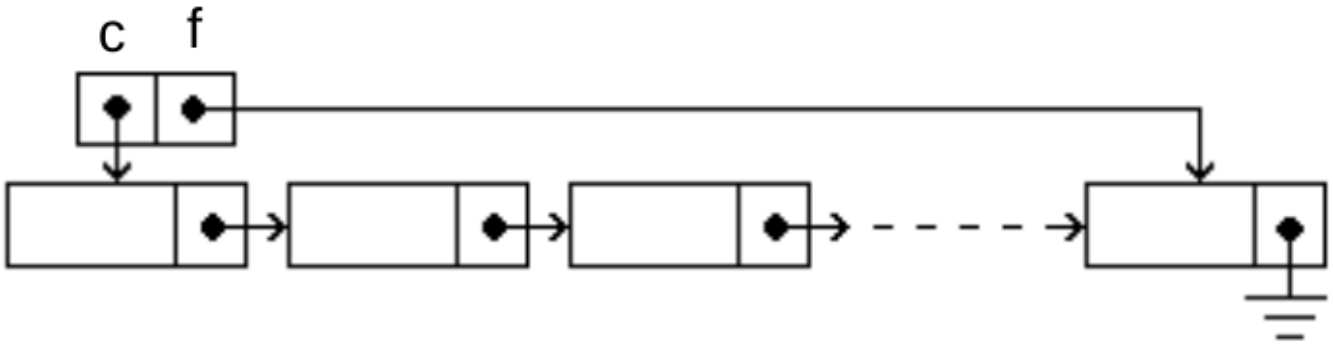


- Para representar uma fila dessa forma, é preciso definir dois TADs:
  - `No`
  - `FilaEncad`

Fila

Encadeamento dos nós

- Representação por encadeamento dos nós.
- Esquematicamente, seja uma fila encadeada com n nós:



- Para representar uma fila dessa forma, é preciso definir dois TADs:
  - `No`
  - `FilaEncad`

TAD Fila Encadeada

- Definição da classe do TAD `No` (No.h)
- Pode-se utilizar o mesmo nó definido anteriormente para uma lista simplesmente encadeada ou pilha encadeada.

```
class No
{
 public:
   // ...

 private:
   float info; // informacao do No
   No *prox;
};
```

TAD Fila Encadeada

- Definição da classe do TAD `FilaEncad`
- Arquivo: FilaEncad.h

```
class FilaEncad
{
 private:
  No *c; // ponteiro para No do comeco
  No *f; // ponteiro para No do fim

 public:
  FilaEncad();                // construtor
  ~FilaEncad();               // destrutor
  float consultaInicio(); // retorna No do inicio
  void entra(float val);  // insere No no fim
  void sai();                 // elimina No do inicio
  bool vazia();               // verifica se esta vazia
};
```

TAD Fila Encadeada

- Arquivo: FilaEncad.cpp

```
FilaEncad::FilaEncad()
{
  c = NULL;
  f = NULL;
}

FilaEncad ::~FilaEncad ()
{
  No *p = c;
  while(c != NULL)
  {
    c = p->consultaProx();
    delete p;
    p = c;
  }
  f = NULL;
}
```

TAD Fila Encadeada

```
bool FilaEncad::vazia()
{
  // verifica se fila esta vazia
  if(c == NULL)
    return(true);
  else
    return(false);
}

float FilaEncad::consultaInicio()
{
  if(c != NULL)
    return c->consultaInfo();
  else
  {
    cout << "Fila vazia!" << endl;
    exit(1);
  }
}
```

TAD Fila Encadeada

```
void FilaEncad::entra(float val);
{
  No *p = new No();
  p->atribInfo(val);
  p->atribProx(NULL);

  if(f == NULL)
    c = p;                // insere o primeiro No
  else
    f->atribProx(p);      // liga No p na fila

  f = p;                  // No p passa a ser o ultimo
}
```

TAD Fila Encadeada

```
void FilaEncad::sai()
{
  No *p;
  if(c != NULL)
  {
    p = c;
    c = p->consultaProx();

    if(c == NULL)
      f = NULL;          // a fila esvaziou

    delete p;            // exclui o No do inicio
  }
  else
  {
    rotina_erro(1);
  }
}
```

Exercícios

1. Desenvolver uma função para inverter o sentido de uma **fila** representada por uma lista encadeada.

2. Considerando uma **pilha** representada por uma lista encadeada onde cada nó possui um valor real, desenvolver um procedimento para desempilhar os seus nós montando uma lista circular (duplamente encadeada).

3. O gerente de desenvolvimento de aplicações de uma empresa observou que diversos programas trabalham com **filas** encadeadas que devem ser gravadas em memória auxiliar. Para isso é necessário representar a fila contiguamente. Desenvolver um procedimento para fazer esta transformação sabendo que cada nó possui um valor real e que uma fila possui no máximo 500 nós.

Exercícios

4. Desenvolver um procedimento para fazer a transformação inversa da anterior.

5. Desenvolver um procedimento para montar duas **filas** contíguas, sendo que uma deve conter somente números inteiros negativos e a outra, os demais números. Os números inteiros serão fornecidos através de uma **pilha** encadeada em que cada nó possui um número inteiro e qualquer. Considerar que a pilha possui no máx. 100 nós.

6. Desenvolver um programa para ler diversos valores inteiros e montar uma **fila** encadeada, usando o TAD FilaEncad e atendendo as seguintes características:
   a) O último valor a ser lido é um FLAG com $X = 0$.
   b) Na fila só podem entrar nós com $X > 0$ e maior que a informação X do último nó já incluído na fila.
   c) Para cada valor $X < 0$ lido, retirar um nó da fila, se não for vazia.
   d) No final, imprimir todos os nós da fila